

An Improved Heuristic Based Genetic Algorithm for Bin Packing Problem

1st Aluísio Cardoso Silva

Department of Computer Science
Federal University of Juiz de Fora
Juiz de Fora, Brazil
Email: aluisio@ice.ufjf.br

2nd Carlos Cristiano Hasenclever Borges

Department of Computer Science
Federal University of Juiz de Fora
Juiz de Fora, Brazil
Email: cchborges@ice.ufjf.br

Abstract—The NP-hard bin packing problem is a widely studied grouping problem that is base to model several useful and practical problems from contexts such as industrial or transportation logistics and machine scheduling. Given the complexity involved in the BPP solution, the use of most straightforward strategies, named building heuristics, provides the desired computational efficiency but, usually, obtaining non-optimal or local minima solutions. Otherwise, adapted metaheuristics to handle the problem allows an effective global search which augments the chance to obtain optimal or quasi-optimal solutions, however with a high computational cost. Modelings that combine both strategies usually present metaheuristics that depend on building heuristics to correct infeasible solutions generated during the optimization. This work develops a heuristic based genetic algorithm aiming to obtain a hybrid approach constructed to explore the best features of each strategy. Unlike usual, the building heuristic is used as a decoder of mapped solutions, ensuring the feasibility. Additionally, in the final model, a search space restriction with special encoding and specific operators is included to enhance the behavior and performance of the hybrid model. Computational experiments using well-established benchmarks for one-dimensional bin packing problem are carried out to compare the versions of the presented hybrid methods with high-quality methods presented in the literature. The results indicate the potential for the presented strategy to solve the one-dimensional bin packing problems.

Index Terms—grouping problems, bin packing, genetic algorithm, building heuristics.

I. INTRODUCTION

Grouping problems are based on the task of partitioning a set of items into mutually disjoint subsets. In formal terms, since S is the set of all items and S_i one of the possible subsets:

$$\cup S_i = S \text{ and } S_i \cap S_k = \emptyset, \text{ where } i \neq k. \quad (1)$$

Derived problems based on grouping problems include bin packing, graph coloring, batch-processing machine scheduling problem, data clustering, etc. [1], [2]. Each problem has specific constraints to define feasible groupings and cost function that evaluates a possible solution associated with the composition of the groups [3].

The authors thank the Coordination for the Improvement of Higher Level Personnel (CAPES) and National Council for Scientific and Technological Development (CNPq) and the State of Minas Gerais Research Support Agency (FAPEMIG) for the financial support for the accomplishment of this paper.

The bin packing problem (BPP) is one of the most studied of this class of problems. Adaptations and variations of the simplest version, the one-dimensional bin packing problem, can be applied to a wide variety of practical problems such as allocation of resources in computers, industrial or transportation logistics and sub-problems of different areas. One-dimensional BPP is defined as follows: given the capacity C of a bin, and a finite number of items N , each with specific weights w , $0 < w_i \leq C$, find the minimum number of the total bins required to package all items, such that the sum of the weights in each bin is at most C . Due to its classification as NP-hard [4], approximate approaches are encouraged to tackle the problem. Exact methods based on mathematical programming are a potential option to solve the problem and, although have a disadvantage of scalability, these methods have been highly improved to address the BPP over the years (see [5]).

In this work, three variations of a genetic algorithm were developed to solve the bin packing problem with representation based on permutation. Similar encodings are found in [6] and [7] but with a different exploration of the potential provided by the approach. The purpose of this paper includes the comparison of the three algorithms presented through the evaluation of the search quality for solving benchmarks problems recognized by the scientific community. Besides, through a comparative analysis of the experimental results, the competitiveness of the proposed model in relation to the methods considered the state-of-art in literature is shown.

In the next section, is presented a brief description of the relevant works referenced as high-quality heuristic and metaheuristic methods for bin packing problem solution.

II. RELATED WORKS

The Grouping Genetic Algorithm (GGA), proposed by Falkenauer [3], is commonly cited as a referential metaheuristic constructed to solve the BPP. The structure of the algorithm is improved in [8] by adding a local optimization procedure. Results obtained by the so-called Hybrid Grouping Genetic Algorithm (HGGA) are used as comparative basis in several papers in the literature, which often result from adaptations to the original proposal of Falkenauer. An example of adaptation is the adoption of different metaheuristics instead of GA, as

in the case of the Grouping Particle Swarm Optimization Algorithm (GPSO) [9] and Grouping Evolution Strategies (GES) [2], that obtained competitive results in relation to the work of Falkenauer. Enhancements and operators highly specialized in BPP were introduced in the basic structure of GGA in GGA-CGT [10], improving the quality of the results. The work described in [11], implements GGA with high computational performance through the graphics processing unit (GPU), called GGA-CUDA, which allows a more efficient exploration of the search space.

Two other heuristic approaches are often cited in related works due to the high quality of results achieved. The first, called HI_BP [12], presents a complex heuristic that combines different techniques previously suggested in other studies, such as search space reduction, upper and lower bounding strategies, a greedy algorithm to find a solution, and an improvement process based on tabu search. The Perturbation-SAWMBS [13] introduces a new concept of sufficient average weight (SAW) to control the average weight of items packed by the MBS (minimum bin slack) building heuristic in each bin. This concept aims to provide a more efficient packing and was incorporated into the original algorithm, Perturbation-MBS, which applies a variable neighborhood search to construct a new solution through perturbations of the current solution.

In the next subsection, the different encodings used in metaheuristics developed for the BPP solution are presented. Besides, the motivation for the approach adopted in this work is discussed.

A. Encodings

The encoding strategy adopted for the bin packing problem is one of the most important factors associated with the algorithm performance. Thus, several approaches have been proposed previously to represent a solution (also called individual or chromosome) in a string, in which characters are commonly denominated as genes.

Number Encoding (NE) is the most straightforward representation for grouping problems, including BPP, where each gene is associated with an item, and the value of the gene indicates to which group the item belongs. In Grouping Encoding (GE), items allocated in the same bin are gathered on neighboring genes of the chromosome, and the insertion of special markers delimits the subsets. A special encoding is introduced in GGA aiming to directly represent the main components of the solution, the groups, unlike other object-oriented encodings. For this purpose, a group part is included in NE and operators only working on that part of the chromosome. The problem of redundancy, defined as the possibility of distinct chromosomes encoding the same solution, was highlighted by Falkenauer as a critical problem of other encodings. However, the representation used in GGA also suffers from this problem, besides requiring the algorithm the ability to handle chromosomes of variable sizes [1], [14].

The Linear Linkage Encoding (LLE) [1] promises to bypass the redundancy problem by eliminating unwanted symmetry of the search space. Computational experiments performed using

operators specialized in this encoding indicate the need for strategy improvements when compared to HGGA [8]. Many of the different representations suggested for BPP require deep and complex modifications in the functioning of the employed metaheuristics, which results in highly specialized algorithms, diffculting the generalization of technique.

Another important characteristic associated with many encodings is the frequent dependence on the adoption of simple building heuristics to correct infeasible solutions or inconsistencies caused by operators during the search process [2], [3], [9], [10]. In this way, items no longer mapped after a recombination, for example, are reinserted into the solution. Examples of algorithms for this purpose are usually based on building heuristics as First Fit (FF), Best Fit (BF) and Next Fit (NF) which, using a greedy approach, find solutions with a minimum quality guarantee in polynomial time [15]. The number of groups, or bins, obtained by those heuristics are directly related to the ordering of items presented to the adopted heuristic.

Hybrid algorithms developed to search for optimal permutations, based on a metaheuristic, with subsequent decoding by a building heuristic, were developed in the work of Reeves [6] and in the work of Gourgand et al. [7]. Reeves demonstrated by experimental results the viability of the strategy and introduced a concept of problem size reduction associated with this approach, that will be discussed in the next section. Gourgand et al. [7] present the advantages of the bin packing problem modeling with the union of a metaheuristic and a building heuristic decoding, standing out the guarantee of feasible solutions using this encoding, the simplicity of the techniques involved, the ease of implementation and generalization to other problems. A framework for implementation of this strategy is formalized, suggesting the possibility of generalization to any evolutionary computation method. The authors proposed the use of local search algorithms combined with the building heuristics FF, BF, and NF. The results showed that, despite the theoretical potential of the approach, there was no guarantee of efficiency in relation to the other algorithms listed.

In the present work, is constructed an encoding model similar to the described, i.e., using a building heuristic to decode the solution. The search strategy is enhanced by dynamic monitoring and control of the formed groups. Besides, a new problem size restriction strategy is applied, which further permits a progressive reduction of the redundancy possibilities, detailed in the sequence of the algorithms description.

III. PROPOSED ALGORITHMS

The development of the proposed evolutionary algorithm for BPP is depicted incrementally to justify the introduction of each component adequately. Initially, in the first version of the algorithm, Heuristic Based Genetic Algorithm (HBGA), was implemented a GA with a similar structure to “GA + FF” suggested by Reeves [6].

In the second algorithm, named the Heuristic Based Genetic Algorithm with Blocking (HBGAwB), the problem size **restriction** was introduced through the concept of item **blocking**.

The idea is based on blocking items of bins that reach the maximum capacity, aiming to exclude those blocked items from the optimization process. This intervention is performed by the blocking operator after the decoding step and occurs at the **chromosome level**, i.e., when a bin reaches its maximum capacity, items are blocked only in the respective chromosome.

By the other hand, the problem size **reduction** proposed in the work of Reeves [6] promote an intervention at the **population level** and is performed with the **deletion** of items. In other words, identifying a filled bin, the items inserted therein are deleted from all the chromosomes of the population and, thus, the diversity is heavily sacrificed. Moreover, the deletion method can guarantee sub-optimal solutions, as admitted by Reeves, since early deleted items may prevent the formation of optimal groupings during the search process.

In both cases it is assumed that a partial solution was found to the problem, but, in the proposed algorithm, the chromosomal blocking employed allows accelerating the search process without necessarily freezing the same partial solutions on the entire population, allowing independent evolution of each chromosome. In this way, populations of reasonable sizes are able to perform a relatively satisfactory exploration of the search space. However, the process can yet be limited by sub-optimality in a special case where locally optimal blockings occur in all individuals in the population.

Finally, the third algorithm, the Heuristic Based Genetic Algorithm with Dynamic Blocking (HBGAwDB), incorporates the concept of unblocking the blocked items, allowing that the optimization process can escape from locally optimum solutions by reversing possible premature blockings in the chromosomes. The implementation details of the algorithms are explained in the following.

A. Heuristic Based Genetic Algorithm (HBGA)

a) Encoding: the chromosomes are formed by permutations of integers from 1 to N , where N is the number of items to be packed.

b) Building Heuristic decoding: the packing is performed by the First-Fit algorithm, which decodes the solution represented in the permutation mapped on the chromosome. The process determines the total of required bins as well as the capacity filled in each bin, information used in the fitness function.

c) Selection: ranking selection is used to select individuals from the current population to apply the other genetic operators and form the next generation. Chromosomes are ordered based on their fitness and cumulatively receive a probability range proportional to their position in the ranking. To determine the selected individual, a number is randomly generated, which indicates the position of the chosen individual.

d) Crossover operator: Partially Matched or Mapped Crossover (PMX), introduced by Goldberg [16] is one of the most commonly used recombination operators on chromosomes that encode permutation problems [17]. Two parents and two positions are randomly selected. The region delimited

by the positions is switched between parents. For the remainder of the chromosome, the original sequence is preserved as much as possible according to a previous mapping. For every two chromosomes randomly selected, recombination occurs with probability p_c , defined as GA parameter. Chromosomes not submitted to recombination are cloned directly to the next population. The process repeats until a new population is generated.

e) Swap Mutation operator: with probability p_m , each individual generated by the crossover operator is submitted to a random change through a swap between two genes randomly selected on the chromosome. The mutation introduces new information in the population and allows the algorithm to escape from a possible stagnation of the solutions.

f) Fitness function: the fitness function adopted is given by the well-accepted expression defined in (2), introduced by Falkenauer and Delchambre [18]. This objective function is capable of distinguishing the efficiency of two solutions with the same number of bins instead of the simplest idea of evaluates the solution only using the number of bins as reference. So the used function allows a refined measurement of the candidate solutions. Thus, during the evolutionary process, this measure facilitates the elimination of unfilled bins of a solution, given the better fitness value of solutions that present a lower level of capacity utilization in some bins. For this purpose, the search is directed for the maximization of the function depicted in (2), where F_i represents the sum of the weights of the items in the i -th bin, C is the capacity of the bins and M the number of bins obtained by the solution:

$$f_{BPP} = \frac{\sum_{i=1}^M (F_i/C)^2}{M} \quad (2)$$

B. Heuristic Based Genetic Algorithm with Blocking (HBGAwB)

a) Encoding with blocking: each chromosome, of size N as the same as the previous algorithm, is associated with an additional blocking vector, of equal size, which is responsible for controlling the blocked genes.

b) Blocking operator: a blocked gene indicates that the associated item belongs to a filled bin, i.e., the sum of the weights of the items equals the capacity C . The blocking process is outlined with two main objectives: (i) to diminish the total number of permutation solutions since blocked items are excluded of the search, which implicates in the restriction of the search space dimension; (ii) to reduce the possibility of redundancy due to the permutation encoding used since blocked items are not subject to eventual rearrangements that generate the redundant solutions. As mentioned in [3], the degree of redundancy of the permutation encoding is associated with the problem size, as well as in other encodings.

Initially, all items receive the value 0 in their respective position of the blocking vector, indicating the “unblocked” status. During the search process, when items grouped in a bin reach the maximum bin capacity, the blocking vector is updated: the position associated with each grouped item is set

to the respective filled bin number. Thus, values distinct of 0 indicate blocked items.

c) Building Heuristic decoding with blocking: similar to the previous algorithm, the FF heuristic performs the packing of the items before the application of the function evaluation. However, the first step before decoding is executed by checking the blocking vector. The largest integer found in this vector indicates the current number of filled bins on the chromosome. Then, the FF algorithm initializes the construction of the remaining bins considering the existence of a predefined number of filled bins indexed by the blocking vector.

During packing, the FF heuristic “jumps” blocked items (already packed) and the building heuristic works only on the currently unblocked items.

d) Swap Mutation operator with blocking: as in the previous algorithm, with probability p_m , the mutation operator applies a swap between two genes randomly selected from the chromosome. However, only the unblocked genes (i.e., associated with the value 0 in the blocking vector) are available for a possible swap.

C. Heuristic Based Genetic Algorithm with Dynamic Blocking (HBGAwDB)

HBGAwB and HBGAwDB share identical structures, except for the introduced operator presented as follows:

a) Unblocking operator: with probability p_u , each chromosome of the population with one or more filled bins has one of those bins randomly chosen to have their items unblocked, reinserting them into the search space. Accordingly, the blocking vector is updated with value 0 for the unblocked items. The unblocking operator is designed to avoid premature blocking, increasing the effectiveness in problems with a complex combination for optimal packing.

D. A Modified First Fit Implementation

The FF building heuristic presents interesting characteristics in the bin packing model composition since its decoding generates viable solutions, i.e., it allows to build the solutions mapped in the permutations encoded respecting the packaging capacity. The blocking strategy developed in conjunction with the FF algorithm can improve the optimization process and population metaheuristics allow efficient exploration of the search space for permutation problems, however, advances in the decoding building heuristics can contribute to enhancing the search, especially avoiding configurations prone to locally optimal solutions.

In this way, the First Fit decoding model was modified objecting to avoid an unnecessary formation of bins with a set of small weight items, which may occur due to disadvantageous sequencing. The idea is to increase the possibility of a reduction of the total number of bins since small items could be important to adjust bins with low capacity available.

The modified First Fit heuristic is constructed using two steps: (i) firstly, the items greater than the half of bins capacity are prepacked in individual bins; (ii) following, the classical

First Fit heuristic is applied to finish the solution composition. Separating items that, due to capacity constraints, could not be allocated in the same bin increases the possibility that smaller items will be grouped with larger items before forming a new bin. This approach was previously used in [10] to generate the initial population with relative success.

IV. EXPERIMENTAL RESULTS

The benchmarks used in the computational experiments are explored by the best works related to BPP. The initial four sets of instances were generated by Falkenauer in [8]. Each of them contains 20 instances, with 120, 250, 500 and 1000 items respectively, in which the item weights vary uniformly between 20 and 100, and the capacity of the bins is equal to 150. The set defined as “Hard10” was proposed in [19] and contains 10 instances considered difficult, mainly because the weights of items varies widely between 20,000 and 35,000 [20]. Each instance has 200 items to be packed in bins with a capacity of 100,000. The last set, called “Hard28” and used in [21], has been considered very difficult by recent works. In this set, there are 28 instances with different amounts of items (160, 180 or 200), weights ranging from 1 to 800, and bin capacity equal to 1000.

GA parameters were configured based on preliminary tests and reference values used in other GAs adapted to BPP in the literature, as follows: population of 100 chromosomes, the number of elite individuals equal to 10, probability of mutation equal to 80% and crossover for HBGA (or unblocking for HBGAwDB) equal to 50%. It is noteworthy that the crossover operator is only used in the HBGA model. The stopping criterion was established in 10,000 generations, or when the global minimum of bins was found. The algorithms were developed in the programming language of MATLAB 9.0.0 software and each problem was executed with five different sequential generating seeds of random numbers, set as “default” language type and starting from 1 as reported in [10], [12], [13]. Therefore, 590 executions were performed for each of the three algorithms.

In Table I, the results of the algorithms implemented in this work are presented. The columns indicate the set of instances (Set); the number of instances of each set (Inst.); the number of instances solved by each algorithm, that is, when the algorithm finds the minimum number of bins (Opts.); the number of executions that found the optimal solution (Opts. runs); the average number of generations to convergence, premature or not (Avg. cvrg.); and the average number of generations until reaching one of the stop criteria (Avg. gens.). The average convergence parameter allows evaluating the speed with which the algorithm stagnated or reached the optimum. The average generations represent directly the effort employed in the searches, considering the defined number of generations.

In the first four instances, in which the item weights are uniformly distributed, the number of optimal solutions found by HBGAwDB was substantially better when compared to the other two algorithms, requiring a drastically less computational effort. The results obtained by HBGAwB in these instances can

TABLE I
RESULTS OBTAINED BY THE PROPOSED ALGORITHMS

Set	Inst.	HBGA				HBGAwB				HBGAwDB			
		Opts.	Opts. runs	Avg. cvrg.	Avg. gens.	Opts.	Opts. runs	Avg. cvrg.	Avg. gens.	Opts.	Opts. runs	Avg. cvrg.	Avg. gens.
u120	20	17	91/100	491	1389	13	78/100	28	2226	20	98/100	233	432
u250	20	14	62/100	1142	4735	13	42/100	67	5839	17	82/100	381	2166
u500	20	11	41/100	4028	8280	3	6/100	118	9411	19	91/100	765	1642
u1000	20	0	0/100	6735	10000	0	0/100	257	10000	20	96/100	2036	2385
hard10	10	8	40/50	941	2857	8	40/50	1189	3083	8	40/50	1334	3175
hard28	28	5	25/140	37	8223	5	24/140	38	8299	5	25/140	42	8227
TOTAL	118	55	259/590	13374	35484	42	191/590	1697	38858	89	431/590	4791	18027

TABLE II
COMPARISON HBGAwDB AND OTHER HIGH-QUALITY ALGORITHMS

Set	HGGA	GPSO	GES	HI_BP	Pert.-SAWMBS	GGA-CGT	GGA-CUDA	HBGAwDB
u120	2	0	0	0	0	0	-	0
u250	2	-	-	0	1	0	-	3
u500	0	-	-	0	0	0	-	1
u1000	0	-	-	0	0	0	-	0
Hard10	-	2	2	0	0	0	-	2
Hard28	-	-	-	23	23	12	23	23

be associated with the problem of sub-optimality highlighted earlier in the previous section and considered as “possibly rare” in the work of Reeves [6]. The algorithm converges prematurely and finds a much smaller number of optimal solutions than the other two models. However, for the hard instances, the algorithms obtained similar results in terms of overall optimal found and numbers of executions that found these values, except for the Hard28 instance, in which the HBGAwB found the optimal objective value in one execution less than the other algorithms. The values associated with the convergence generations and the effort employed by the algorithms were very similar in these evaluated instances.

Table 2 shows the comparison between the proposed algorithm that presented the best overall result, HBGAwDB, and other algorithms highlighted among the best heuristic and metaheuristic methods used in the BPP solution, all referenced in the related works section. For each class of problems treated, the number of instances not solved by the algorithms is exposed, and a “dash” indicates that the work did not present experiments for the indicated set of instances.

Our algorithm was able to find all the solutions for the “u120” and “u1000” classes, behavior similar to part of the other algorithms. Some preliminary tests revealed that increasing in 30% the maximum number of generations promotes a reduction in the number of responses not found in the “u250” and “u500” classes (reaching 1 and 0, respectively), but the value was set to 10,000 due to be considered as a maximum limit for GAs adapted to BPP in [10]. It is important to notice that the average number of generations needed until the response was found, or until the stagnation of optimization, varies in the range of 42 to 2036, indicating that the obtained results do not require all the effort employed. In the hard instances, the proposed algorithm presented similar quality, in general, to the other algorithms listed. It is worth to emphasize

especially the Hard28 instance, which is strongly encouraged by the community as the benchmark to be tested for high-quality algorithms. In this class, the presented algorithm was able to find the same number of optimal solutions pointed out by two of the three best algorithms and the recent GGA-CUDA.

V. CONCLUSIONS

A hybrid heuristic construction based on a genetic algorithm is presented in this work to solve the well know bin packing problem efficiently. The main idea is to obtain a strategy that presents potential to circumvent the sub-optimal solution obtained by building heuristics using the global search provided by optimization metaheuristics. Besides, this type of construction has the advantage of generating only feasible solutions.

The incorporation of a blocking strategy to the first proposed model indicates an interesting way to improve the search by limiting the search space of the optimization process, consequently, reducing during the evolution, the possibilities of redundancy for the used encoding. Furthermore, an unblocking operator that allows a dynamic blocking complete the final strategy generating an efficient hybrid method for bin packing problem solution. The numerical experiments which compare the proposed methods with the state-of-art algorithms indicate the potential for this kind of hybrid method, mainly when specific operators are designed to improve the global search and to avoid sub-optimal solutions, the main characteristic of building heuristics.

Finally, it is important to stress out that further improvements are necessary to become the method completely competitive. These improvements possibly will follow the same pattern of the proposed hybrid model by considering other building heuristics to replace the First Fit heuristic adopted.

Besides, a better comprehension of the packing mechanism in relation to local optimums obtainment could indicate a direction for the construction of more effective operators to improve the performance of the hybrid metaheuristic.

REFERENCES

- [1] Ö. Ülker, E. E. Korkmaz, and E. Özcan, "A grouping genetic algorithm using linear linkage encoding for bin packing," in *International Conference on Parallel Problem Solving from Nature*. Springer, 2008, pp. 1140–1149.
- [2] A. H. Kashan, A. A. Akbari, and B. Ostadi, "Grouping evolution strategies: An effective approach for grouping problems," *Applied Mathematical Modelling*, vol. 39, no. 9, pp. 2703–2720, 2015.
- [3] E. Falkenauer, "A new representation and operators for genetic algorithms applied to grouping problems," *Evolutionary computation*, vol. 2, no. 2, pp. 123–144, 1994.
- [4] M. R. Garey and D. S. Johnson, "Computers and intractability: A guide to the theory of np-completeness," 1979.
- [5] M. Delorme, M. Iori, and S. Martello, "Bin packing and cutting stock problems: Mathematical models and exact algorithms," *European Journal of Operational Research*, vol. 255, no. 1, pp. 1–20, 2016.
- [6] C. Reeves, "Hybrid genetic algorithms for bin-packing and related problems," *Annals of Operations Research*, vol. 63, no. 3, pp. 371–396, 1996.
- [7] M. Gourgand, N. Grangeon, and N. Klement, "An analogy between bin packing problem and permutation problem: A new encoding scheme," in *IFIP International Conference on Advances in Production Management Systems*. Springer, 2014, pp. 572–579.
- [8] E. Falkenauer, "A hybrid grouping genetic algorithm for bin packing," *Journal of heuristics*, vol. 2, no. 1, pp. 5–30, 1996.
- [9] A. H. Kashan, M. H. Kashan, and S. Karimiyan, "A particle swarm optimizer for grouping problems," *Information Sciences*, vol. 252, pp. 81–95, 2013.
- [10] M. Quiroz-Castellanos, L. Cruz-Reyes, J. Torres-Jimenez, C. Gómez, H. J. F. Huacuja, and A. C. Alvim, "A grouping genetic algorithm with controlled gene transmission for the bin packing problem," *Computers & Operations Research*, vol. 55, pp. 52–64, 2015.
- [11] S. O. Ozcan, T. Dokeroglu, A. Cosar, and A. Yazici, "A novel grouping genetic algorithm for the one-dimensional bin packing problem on gpu," in *International Symposium on Computer and Information Sciences*. Springer, 2016, pp. 52–60.
- [12] A. C. Alvim, C. C. Ribeiro, F. Glover, and D. J. Aloise, "A hybrid improvement heuristic for the one-dimensional bin packing problem," *Journal of Heuristics*, vol. 10, no. 2, pp. 205–229, 2004.
- [13] K. Fleszar and C. Charalambous, "Average-weight-controlled bin-oriented heuristics for the one-dimensional bin-packing problem," *European Journal of Operational Research*, vol. 210, no. 2, pp. 176–184, 2011.
- [14] N. Mohamadi, "Application of genetic algorithm for the bin packing problem with a new representation scheme," *Mathematical Sciences*, vol. 4, no. 3, pp. 253–266, 2010.
- [15] D. S. Johnson, "Near-optimal bin packing algorithms," Ph.D. dissertation, Massachusetts Institute of Technology, 1973.
- [16] D. E. Goldberg, R. Lingle *et al.*, "Alleles, loci, and the traveling salesman problem," in *Proceedings of an international conference on genetic algorithms and their applications*, vol. 154. Lawrence Erlbaum, Hillsdale, NJ, 1985, pp. 154–159.
- [17] R. Kumar, G. Gopal, and R. Kumar, "Novel crossover operator for genetic algorithm for permutation problems," *International Journal of Soft Computing and Engineering (IJSCE)*, vol. 3, no. 2, pp. 252–258, 2013.
- [18] E. Falkenauer and A. Delchambre, "A genetic algorithm for bin packing and line balancing," in *Proceedings 1992 IEEE International Conference on Robotics and Automation*. IEEE, 1992, pp. 1186–1192.
- [19] A. Scholl, R. Klein, and C. Jürgens, "Bison: A fast hybrid procedure for exactly solving the one-dimensional bin packing problem," *Computers & Operations Research*, vol. 24, no. 7, pp. 627–645, 1997.
- [20] J. Pérez, R. d. I. Rosa, H. Castillo, and D. Vilarino, "A storage pattern-based heuristic algorithm for solving instances of hard28 datasets for the bin packing problem," *Computación y Sistemas*, vol. 22, no. 1, pp. 235–244, 2018.
- [21] J. E. Schoenfeld, "Fast, exact solution of open bin packing problems without linear programming," *Draft, US Army Space and Missile Defense Command, Huntsville, Alabama, USA*, 2002.